基于局部交换搜索的 BOLT ExtTSP 布局优化

51290713140 赵子祺

说明：本报告根据项目实际终端日志和已展示的补丁代码整理。所有数字都来自本次实验；未完成的验证步骤会明确标注。

摘要

本文在 LLVM BOLT 现有 ExtTSP 基本块排序算法之后，加入一层可选的、有界局部交换搜索（Local Refinement）。算法在固定函数入口块的前提下，尝试小范围交换基本块位置，使用 BOLT 现有的 ExtTSP 评分函数评估候选布局；只有超过正收益阈值的交换才被接受。功能默认关闭，可通过命令行参数启用，修改集中于 ReorderAlgorithm.cpp。

在 branchy 测试负载中，GCC -O2 和 -O3 配置各产生 2 次被接受的交换，ExtTSP 评分增加约 975.82（相对增幅约 0.00247%）；-O1 未发生交换。三个编译配置的原始程序与重写程序输出一致。已在 Ubuntu 24.04 Docker 运行镜像中复现评分改进和输出一致性。运行时间与优化器耗时测试出现较大波动，尚无足够证据证明稳定加速或无性能回退。

1. 项目背景与目标

1.1 背景

BOLT 是面向机器码的后链接优化工具，可利用执行剖析数据对函数与基本块布局进行重新安排。ExtTSP 基于控制流图的执行频率、分支类型和跳转距离估算布局质量，以启发式方式构造排序。启发式结果不必是所有邻近交换操作意义下的局部最优。

1.2 目标与边界

本项目的目标是实现一种简单、可解释、可开关的后处理搜索：在不改变 ExtTSP 原有生成逻辑的前提下，只对小型 CFG 的最终候选布局进行局部改进。代码需要可编译、能够验证基本功能正确性，并留下可复现的源码补丁、运行镜像和实验记录。

边界：本实现优化的是 ExtTSP 代理评分，而不是直接优化真实硬件周期。默认关闭有助于维持原版行为；开启状态下不能仅凭评分非下降就推断实际性能不回退。

2. 环境与源码版本

项目	已使用的配置
源码基线	74c9938e2b57c708900db28348d8fc9fc0604eb9（2026-11-02）
开发分支	course-exttsp-refine
系统 / 虚拟化	Ubuntu 24.04，VMware 虚拟机
LLVM / BOLT	LLVM 24.0.0git，Debug with assertions，X86 目标
构建方式	CMake + Ninja，cmake --build build-debug --target llvm-bolt --parallel 3
修改范围	bolt/lib/Passes/ReorderAlgorithm.cpp

Docker 镜像	bolt-exttsp-refine:20261102；基于 ubuntu:24.04 的预编译运行镜像

3. 算法设计

3.1 搜索流程

① 先运行官方 codelayout::computeExtTspLayout，获得原始基本块排列；② 若未开启选项或基本块数不在 [4,64]，跳过；③ 计算当前布局的 ExtTSP 分数；④ 最多执行两轮，从排列下标 1 开始枚举，与后方最多 8 个位置尝试交换；⑤ 利用 calcExtTspScore 计算候选分数，选取满足门槛的最好候选；⑥ 接受交换后继续搜索，若一轮未改变则提前终止。

3.2 约束与参数

约束	实现规则及作用
开关	-bolt-ext-tsp-local-refine，默认 false
函数规模	4 ≤ 基本块数 ≤ 64；避免大函数产生过高搜索开销
入口块	排列下标 0 不参与交换；保护函数入口顺序
搜索窗口	SwapWindow = 8；控制局部交换范围
搜索轮数	MaxPasses = 2；限制迭代工作量
接受条件	CandidateScore > BestScore + 1.0
数值有效性	仅在评分为有限数时搜索；候选评分也需有限

3.3 复杂度与适用性

每次候选交换都调用全量 ExtTSP 评分函数。设基本块数量为 N，CFG 边数量为 E，搜索窗口 W=8，轮数 P≤2，则一次搜索的评分计算工作量约为 O(P·N·W·(N+E))。由于 N≤64，本原型以可解释性和实现简单为优先；尚未对局部增量评分做专门优化。

3.4 实现位置

在 ExtTSPReorderAlgorithm::reorderBasicBlocks 中，获得 computeExtTspLayout 的 Result 之后、将 Result 转成最终 BasicBlockOrder 之前执行局部交换。复用现有 BlockSizes / JumpCounts 输入和 codelayout::calcExtTspScore，避免修改 LLVM 共享的 CodeLayout 算法实现。

```
// 核心接受规则（摘取）
const double MinGain = 1.0;
if (std::isfinite(CandidateScore) &&
    CandidateScore > BestScore + MinGain) {
  BestScore = CandidateScore;
  BestJ = J;
}
// 仅接受可测的正收益；入口索引 0 不参与交换
```

4. 正确性和算法效果实验

4.1 测试程序

实验使用 branchy.c：循环执行由伪随机状态驱动的 16 分支分派，其中前 4 个分支被更频繁地选择，形成不均匀冷热分布。通过 BOLT -instrument 生成 branchy.fdata，然后分别关闭和开启局部优化，使用相同 Profile 运行 ExtTSP（-tsp-threshold=0）。

4.2 测试结果

编译配置	被接受交换	ExtTSP 分数增加	输出比较
GCC -O1	0	0	通过
GCC -O2	2	约 975.82	通过
GCC -O3	2	约 975.82	通过

O2 / O3 开启优化后的典型日志：

```
BOLT-LOCAL-REFINE: swaps=2 score_before=3.944552e+07 score_after=3.944650e+07 gain=9.758199e+02
```

日志中的 before/after 为科学计数法截断显示，精确增益以 gain=9.758199e+02 为准。按 before 的近似值计算，相对评分改进约为 0.00247%。O1 和 O2/O3 的结果不能视作完全独立的三份工作负载；这些只是一个测试程序的不同编译形态。

5. 性能与开销评估

5.1 重写程序执行时间

在 VMware 中，首先进行 40 轮交替测量，得到 ExtTSP 97.2161 ms 与 ExtTSP+Refine 98.8857 ms 的中位数；但前 20 轮与后 20 轮方向相反，bootstrap 变化区间跨过 0。第二轮成组测试将进程绑定到一颗虚拟 CPU，每轮每版本执行 10 次，共 30 轮，结果如下。

指标	结果
ExtTSP 时间中位数	107.3028 ms
ExtTSP+Refine 时间中位数	109.5151 ms
成对相对变化中位数	+0.0060%（正值代表 Refine 更慢）
Refine 更快轮次	15 / 30
前半段成对变化中位数	-2.6614%
后半段成对变化中位数	+0.5295%

两种版本的总体时间中位数不等同于成对差值中位数。数据存在明显的时序漂移和波动，因此不能据此推断稳定加速，也不能把个别负值或正值当成可靠的真实性能结论。

5.2 BOLT 自身处理时间

指标	结果
关闭 Local Refinement	67.593 ms（10 轮中位数）
开启 Local Refinement	110.490 ms（10 轮中位数）
两中位数相对变化	+63.462%（仅为本次端到端观测）

各轮耗时跨度较大：例如第 1 轮 OFF=65.08 ms / ON=289.63 ms，而第 5 轮 OFF=165.78 ms / ON=61.52 ms。两者受系统调度、文件缓存、进程启动等因素干扰，无法将 +63.462% 直接归因于局部搜索算法。要精确量化算法本身成本，应在优化 pass 内增加计时器，并使用更大负载或更稳定的测试平台。

5.3 性能结论

当前证据支持“局部搜索能够改进某些函数的 ExtTSP 代理评分，且已验证基础输出一致”；不支持“稳定提高运行速度”或“启用时绝无性能回退”的表述。实际部署建议保持默认关闭，仅在实验和经验证的工作负载上主动开启。

6. Docker 复现与交付

6.1 运行镜像的性质

已构建 bolt-exttsp-refine:20261102 镜像，Dockerfile 使用 ubuntu:24.04 基础镜像，复制预编译 Debug llvm-bolt、branchy、branchy.c、branchy.fdata 与实验记录。该镜像不在构建期间从 LLVM 源码重新编译 BOLT，不能替代源码可重建性证据。因此还须配套固定 Git SHA、补丁和 README。

6.2 Docker 内已验证实验

```
docker run --rm bolt-exttsp-refine:20261102 bash -lc '
"  llvm-bolt ./branchy -o /tmp/branchy.refine -data=/work/branchy.fdata "
"    -reorder-blocks=ext-tsp -tsp-threshold=0 -reorder-functions=none "
"    -bolt-ext-tsp-local-refine=true && "
"  diff -u <(./branchy) <(/tmp/branchy.refine)"
"""
```

实际容器日志显示 swaps=2、gain=9.758199e+02，随后输出“PASS: Docker 内优化及正确性测试通过”。

6.3 镜像导出

已导出 ~/bolt-verify/deliverables/bolt-exttsp-refine-20261102.tar.gz，ls -lh 显示约 558 MB。该文件在当前对话中仅由用户 Ubuntu 终端报告；本报告未直接读取或校验其内部字节。建议提交前执行 gzip -t、sha256sum，并保留校验结果。

7. 已知限制与改进方向

（1）当前测试主要集中于 branchy 单一负载，编译选项变化不能替代多应用验证。

（2）全量评分计算的开销有待使用 pass 级计时与更大 CFG 进行评估。

（3）评分增益很小，且硬件性能不一定与 ExtTSP 代理评分一致。

（4）正确性目前以输出比较为主，建议进一步补充 LLVM/BOLT 回归测试、不同控制流形态及异常路径测试。

（5）默认关闭时应补做与官方基线的二进制行为一致性检查；Debug 构建可能带来元数据或时间戳差异，比较时应避免把文件哈希差异直接视为布局变化。